

Sprint 1 — Gamer Backlog API

API REST para gestionar y priorizar tu backlog de videojuegos

Stack: Node.js · Express · Prisma · MySQL (Aiven) · Zod · ES Modules

Índice

- 1. [Resumen del proyecto](#)
- 2. [Stack tecnológico](#)
- 3. [Estructura del proyecto](#)
- 4. [Configuración inicial](#)
- 5. [Modelo de datos](#)
- 6. [Endpoints de la API](#)
- 7. [Algoritmo de priorización](#)
- 8. [Validación con Zod](#)
- 9. [Manejo de errores](#)
- 10. [Pruebas con Bruno](#)
- 11. [Guía paso a paso](#)
- 12. [Capturas recomendadas](#)

1. Resumen del proyecto

Backend para una aplicación que permite a gamers gestionar su backlog de videojuegos. Los usuarios pueden registrar juegos con datos como categoría, etiquetas, puntuación Metacritic y horas estimadas. La app aplica un algoritmo de priorización (puntuación / horas) para sugerir qué jugar primero.

Sprint 1 es una API para un único gamer, sin autenticación. CRUD completo con filtros, ordenación y validación Zod.

2. Stack tecnológico

Tecnología	Versión	Uso
Node.js	24	Entorno de ejecución
Express	4.21	Framework web
Prisma	6.19	ORM para base de datos
MySQL	8	Base de datos relacional (Aiven cloud)
Zod	3.25	Validación de esquemas
ES Modules	nativo	import/export sin transpilación

3. Estructura del proyecto

```
gamer-backlog-api/  
├─ docs/
```

```

└─ bruno/                                ← Colección de Bruno para pruebas
    └─ Health/
        └─ Health check.bru
    └─ Games/
        └─ Crear juego.bru
        └─ Listar juegos.bru
        └─ Listar con filtros.bru
        └─ Obtener juego por ID.bru
        └─ Editar juego.bru
        └─ Eliminar juego.bru
        └─ Marcar como completado.bru
        └─ Probar validacion.bru
└─ prisma/
    └─ schema.prisma                    ← Modelo de datos
    └─ migrations/                      ← Migraciones SQL autogeneradas
└─ src/
    └─ config/
        └─ env.js                      ← Variables de entorno
        └─ prisma.js                  ← Cliente Prisma singleton
    └─ docs/                           ← Documentación técnica
        └─ api-spec.md                ← Especificación de endpoints
        └─ architecture.md            ← Decisiones técnicas
        └─ data-model.md              ← Modelo de datos
        └─ Sprint1.md                 ← ← Este documento
    └─ middlewares/
        └─ error.middleware.js         ← Captura ZodError + errores genéricos
        └─ not-found.middleware.js     ← 404 para rutas inexistentes
    └─ modules/
        └─ games/
            └─ games.controller.js     ← Handlers HTTP
            └─ games.service.js        ← Lógica de negocio
            └─ games.schemas.js       ← Schemas Zod
            └─ games.routes.js         ← Definición de rutas
    └─ routes/
        └─ index.js                   ← Router principal
    └─ utils/
        └─ priority.js                ← Algoritmo de priorización
    └─ app.js                         ← Configuración Express
    └─ server.js                      ← Punto de entrada
└─ .env                              ← Variables de entorno (gitignored)
└─ .env.example                      ← Plantilla para otros desarrolladores
└─ .gitignore
└─ AGENTS.md                        ← Contexto para asistentes IA
└─ package.json
└─ README.md

```

4. Configuración inicial

4.1 Variables de entorno

Archivo `.env` en la raíz del proyecto:

```
PORT=4000
NODE_ENV=development
DATABASE_URL="mysql://usuario:password@host:puerto/defaultdb?ssl-mode=REQUIRED"
```

4.2 Express app

src/app.js — Configuración del servidor Express con middlewares globales:

```
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';

import { router } from './routes/index.js';
import { errorMiddleware } from './middlewares/error.middleware.js';
import { notFoundMiddleware } from './middlewares/not-found.middleware.js';

export const app = express();

app.use(helmet());           // Seguridad HTTP
app.use(cors());             // CORS
app.use(express.json());     // Parsear JSON bodies
app.use(morgan('dev'));      // Logging HTTP

app.use('/api', router);     // Rutas principales

app.use(notFoundMiddleware); // 404
app.use(errorMiddleware);    // Errores
```

4.3 Prisma client

src/config/prisma.js — Singleton del cliente Prisma:

```
import { PrismaClient } from '@prisma/client';
export const prisma = new PrismaClient();
```

5. Modelo de datos

5.1 Schema Prisma

prisma/schema.prisma :

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}
```

```

}

model Game {
  id          Int          @id @default(autoincrement())
  name        String
  category    String
  tags        String      @db.Text
  metacriticScore Int
  hoursToBeat Float
  completed    Boolean     @default(false)
  completedAt  DateTime?
  notes        String?     @db.Text
  rating       Int?
  priorityScore Float      @default(0)
  createdAt    DateTime    @default(now())
  updatedAt    DateTime    @updatedAt
}

```

5.2 Descripción de campos

Campo	Tipo	Descripción
id	Int (PK, autoincrement)	Identificador único del juego
name	String	Nombre del juego
category	String	Categoría (RPG, Shooter, Aventura, Indie...)
tags	Text	Etiquetas separadas por comas (ej: "accion,zombies,estrategia")
metacriticScore	Int (0-100)	Puntuación en Metacritic
hoursToBeat	Float	Horas estimadas para completar el juego
completed	Boolean	Indica si el juego está completado
completedAt	DateTime?	Fecha en que se completó (se asigna automáticamente)
notes	Text?	Notas personales sobre el juego
rating	Int? (1-5)	Valoración personal del juego
priorityScore	Float	Prioridad calculada (metacriticScore / hoursToBeat)
createdAt	DateTime	Fecha de creación (automática)
updatedAt	DateTime	Fecha de última modificación (automática)

5.3 Migración

Para crear la tabla en MySQL:

```
npx prisma migrate dev --name init
```

Esto genera un archivo SQL en `prisma/migrations/` y lo ejecuta contra la base de datos.

6. Endpoints de la API

6.1 GET /api/health

Health check para verificar que el servidor funciona.

Respuesta (200):

```
{
  "success": true,
  "data": { "status": "ok" }
}
```

6.2 GET /api/games — Listar juegos con filtros y ordenación

Parámetros query (todos opcionales):

Parámetro	Valores	Descripción
search	string	Búsqueda parcial por nombre del juego
category	string	Filtro por categoría exacta
tag	string	Búsqueda parcial en etiquetas
completed	"true" / "false"	Filtrar por estado de completado
sortBy	"priorityScore", "metacriticScore", "hoursToBeat", "name"	Campo por el que ordenar
order	"asc" / "desc"	Dirección de ordenación

Ejemplos de uso:

```
GET /api/games
GET /api/games?category=Aventura
GET /api/games?search=zelda
GET /api/games?completed=false&sortBy=priorityScore&order=desc
GET /api/games?tag=accion&sortBy=name&order=asc
```

Respuesta (200):

```
{
  "success": true,
  "data": [
    {
```

```

    "id": 2,
    "name": "Journey",
    "category": "Indie",
    "tags": "aventura,arte",
    "metacriticScore": 92,
    "hoursToBeat": 3,
    "completed": true,
    "completedAt": "2026-05-28T16:15:41.446Z",
    "notes": "Increíble experiencia",
    "rating": 5,
    "priorityScore": 30.67,
    "createdAt": "2026-05-28T16:15:40.793Z",
    "updatedAt": "2026-05-28T16:15:41.447Z"
  }
]
}

```

Implementación en service:

```

async listGames(filters = {}) {
  const where = {};

  if (filters.search) {
    where.name = { contains: filters.search };
  }

  if (filters.category) {
    where.category = filters.category;
  }

  if (filters.tag) {
    where.tags = { contains: filters.tag };
  }

  if (filters.completed !== undefined) {
    where.completed = filters.completed === 'true';
  }

  const orderBy = {};
  const sortBy = filters.sortBy || 'priorityScore';
  const order = filters.order || 'desc';
  orderBy[sortBy] = order;

  const games = await prisma.game.findMany({ where, orderBy });

  return games.map((game) => ({
    ...game,
    completedAt: game.completedAt?.toISOString() ?? null,
  }));
}

```

6.3 GET /api/games/:id — Obtener juego por ID

Respuesta (200): Objeto del juego. Respuesta (404):

```
{
  "success": false,
  "message": "Juego no encontrado"
}
```

6.4 POST /api/games — Crear juego

Crea un nuevo juego y calcula automáticamente su `priorityScore` .

Body:

```
{
  "name": "The Legend of Zelda: Breath of the Wild",
  "category": "Aventura",
  "tags": "accion,aventura,mundo-abierto",
  "metacriticScore": 97,
  "hoursToBeat": 55,
  "completed": false,
  "notes": "",
  "rating": null
}
```

Respuesta (201): Objeto del juego creado con `priorityScore` calculado.

Validación Zod:

```
export const createGameSchema = z.object({
  name: z.string().min(1, 'El nombre es obligatorio'),
  category: z.string().min(1, 'La categoría es obligatoria'),
  tags: z.string().min(1, 'Añade al menos una etiqueta'),
  metacriticScore: z.number().int().min(0).max(100),
  hoursToBeat: z.number().positive(),
  completed: z.boolean().optional().default(false),
  notes: z.string().optional(),
  rating: z.number().int().min(1).max(5).optional(),
});
```

6.5 PUT /api/games/:id — Actualizar juego

Actualiza los campos enviados (todos opcionales). Si se modifican `metacriticScore` o `hoursToBeat` , recalcula `priorityScore` .

Body (parcial):

```
{
  "name": "Zelda Breath of the Wild",
```

```
"metacriticScore": 95,
"hoursToBeat": 50
}
```

Respuesta (200): Objeto del juego actualizado. **Respuesta (404):** Mensaje de juego no encontrado.

Cálculo de prioridad en update:

```
async updateGame(id, input) {
  const existing = await prisma.game.findUnique({ where: { id } });

  if (!existing) {
    return null;
  }

  const metacriticScore = input.metacriticScore ?? existing.metacriticScore;
  const hoursToBeat = input.hoursToBeat ?? existing.hoursToBeat;
  const priorityScore = calculatePriorityScore(metacriticScore, hoursToBeat);

  const game = await prisma.game.update({
    where: { id },
    data: { ...input, priorityScore },
  });

  return { ...game, completedAt: game.completedAt?.toISOString() ?? null };
}
```

6.6 DELETE /api/games/:id — Eliminar juego

Respuesta (200):

```
{
  "success": true,
  "data": { "message": "Juego eliminado correctamente" }
}
```

Respuesta (404): Mensaje de juego no encontrado.

6.7 PATCH /api/games/:id/complete — Marcar como completado

Marca el juego como completado. Establece `completed: true` y `completedAt` con la fecha actual automáticamente. Acepta notas y valoración opcionales.

Body (opcional):

```
{
  "notes": "Una obra maestra absoluta",
  "rating": 5
}
```


Respuesta (200): Objeto del juego con `completed: true` y `completedAt` con la fecha actual.

Validación Zod:

```
export const completeGameSchema = z.object({
  notes: z.string().optional(),
  rating: z.number().int().min(1).max(5).optional(),
});
```

Implementación en service:

```
async completeGame(id, data = {}) {
  const existing = await prisma.game.findUnique({ where: { id } });

  if (!existing) {
    return null;
  }

  const game = await prisma.game.update({
    where: { id },
    data: {
      completed: true,
      completedAt: new Date(),
      ...(data.notes !== undefined && { notes: data.notes }),
      ...(data.rating !== undefined && { rating: data.rating }),
    },
  });

  return { ...game, completedAt: game.completedAt?.toISOString() ?? null };
}
```

7. Algoritmo de priorización

El corazón de la aplicación. Es una fórmula simple pero efectiva:

```
priorityScore = metacriticScore / hoursToBeat
```

Fundamento: Priorizamos juegos con alta puntuación y pocas horas para dar una "sobredosis de dopamina instantánea" — terminas rápido, te sientes bien y pasas al siguiente.

Ejemplos:

Juego	Metacritic	Horas	Prioridad
Journey	92	3	30.67 ← ¡Este primero!
Zelda BOTW	97	55	1.76
The Witcher 3	93	100	0.93

Implementación:

```
// src/utils/priority.js
export function calculatePriorityScore(metacriticScore, hoursToBeat) {
  if (hoursToBeat <= 0) {
    return 0;
  }

  return Number((metacriticScore / hoursToBeat).toFixed(2));
}
```

8. Validación con Zod

Todos los endpoints validan los datos de entrada con Zod antes de procesarlos.

8.1 Schemas

```
// src/modules/games/games.schemas.js

// Creación: todos los campos obligatorios
export const createGameSchema = z.object({
  name: z.string().min(1, 'El nombre es obligatorio'),
  category: z.string().min(1, 'La categoría es obligatoria'),
  tags: z.string().min(1, 'Añade al menos una etiqueta'),
  metacriticScore: z.number().int().min(0).max(100),
  hoursToBeat: z.number().positive(),
  completed: z.boolean().optional().default(false),
  notes: z.string().optional(),
  rating: z.number().int().min(1).max(5).optional(),
});

// Actualización: todos opcionales (partial)
export const updateGameSchema = createGameSchema.partial();

// Completar: solo notas y rating
export const completeGameSchema = z.object({
  notes: z.string().optional(),
  rating: z.number().int().min(1).max(5).optional(),
});

// Query params: filtros y ordenación
export const listGamesQuerySchema = z.object({
  search: z.string().optional(),
  category: z.string().optional(),
  tag: z.string().optional(),
  completed: z.enum(['true', 'false']).optional(),
  sortBy: z
    .enum(['priorityScore', 'metacriticScore', 'hoursToBeat', 'name'])
    .optional()
    .default('priorityScore'),
});
```

```
    order: z.enum(['asc', 'desc']).optional().default('desc'),
  });
```

8.2 Errores de validación

Cuando Zod detecta datos inválidos, el middleware de errores captura el `ZodError` y devuelve:

```
{
  "success": false,
  "message": "Error de validación",
  "errors": [
    { "field": "name", "message": "El nombre es obligatorio" },
    { "field": "category", "message": "Required" },
    { "field": "tags", "message": "Required" },
    { "field": "metacriticScore", "message": "Required" },
    { "field": "hoursToBeat", "message": "Required" }
  ]
}
```

9. Manejo de errores

9.1 Error middleware

```
// src/middlewares/error.middleware.js
import { ZodError } from 'zod';

export const errorMiddleware = (err, _req, res, _next) => {
  // Errores de validación Zod → 400 con detalles
  if (err instanceof ZodError) {
    const errors = err.errors.map((e) => ({
      field: e.path.join('.'),
      message: e.message,
    }));

    return res.status(400).json({
      success: false,
      message: 'Error de validación',
      errors,
    });
  }

  // Errores genéricos
  console.error(err);

  res.status(err.status ?? 500).json({
    success: false,
    message: err.message ?? 'Internal server error',
  });
};
```

9.2 404 middleware

```
// src/middlewares/not-found.middleware.js
export const notFoundMiddleware = (_req, res) => {
  res.status(404).json({ success: false, message: 'Route not found' });
};
```

10. Pruebas con Bruno

10.1 Importar la colección

1. Abre Bruno
2. **File** → **Import Collection** → selecciona la carpeta `docs/bruno/`
3. Arranca el servidor con `npm run dev`

10.2 Requests disponibles

Colección: Gamer Backlog API

- ├ Health
 - └ Health check GET /api/health
- └ Games
 - ├ Crear juego POST /api/games
 - ├ Listar juegos GET /api/games
 - ├ Listar con filtros GET /api/games?category=...
 - ├ Obtener juego por ID GET /api/games/1
 - ├ Editar juego PUT /api/games/1
 - ├ Eliminar juego DELETE /api/games/1
 - ├ Marcar como completado PATCH /api/games/1/complete
 - └ Probar validacion POST /api/games (body vacío → error 400)

11. Guía paso a paso

Paso 1: Inicializar el proyecto

```
npm init -y
npm install express cors helmet morgan dotenv @prisma/client prisma zod
```

Configurar `package.json` con `"type": "module"` para ES modules.

Paso 2: Configurar Prisma

```
npx prisma init
```

Configurar `prisma/schema.prisma` con el modelo `Game` y MySQL como datasource.

Paso 3: Crear la base de datos en Aiven

1. Crear cuenta en Aiven.io

2. Crear servicio MySQL (plan gratuito)
3. Obtener la URL de conexión
4. Configurar `.env` con `DATABASE_URL`

Paso 4: Migrar la base de datos

```
npx prisma generate
npx prisma migrate dev --name init
```

Paso 5: Crear la estructura del proyecto

```
src/
  config/          → env.js, prisma.js
  middlewares/     → error.middleware.js, not-found.middleware.js
  modules/games/   → controller, service, schemas, routes
  routes/          → index.js
  utils/           → priority.js
  app.js           → Express setup
  server.js        → Entry point
```

Paso 6: Implementar el algoritmo de prioridad

Crear `src/utils/priority.js` con la función `calculatePriorityScore`.

Paso 7: Implementar el CRUD

En orden:

1. **Schemas** — Definir validaciones Zod
2. **Service** — Lógica de negocio con Prisma
3. **Controller** — Handlers HTTP que validan y responden
4. **Routes** — Mapear endpoints a handlers

Paso 8: Configurar middlewares globales

Error handler (captura `ZodError`) y 404 handler.

Paso 9: Verificar

```
npm run dev
```

Probar con Bruno todos los endpoints.

12. Capturas recomendadas

Para tu PDF, estas son las capturas que te recomiendo hacer:

12.1 Estructura del proyecto

- Árbol del proyecto completo (`find . -not -path '*/node_modules/*'`)
- Contenido de `src/` mostrando los módulos

12.2 Base de datos

- Tabla `Game` en MySQL Workbench (`DESCRIBE Game;` `SELECT * FROM Game;`)
- Migraciones en `prisma/migrations/`

12.3 Código clave

- `prisma/schema.prisma` — Modelo de datos
- `src/utils/priority.js` — Algoritmo de priorización
- `src/modules/games/games.schemas.js` — Validaciones Zod
- `src/modules/games/games.service.js` — Lógica de negocio
- `src/middlewares/error.middleware.js` — Manejo de errores

12.4 Pruebas con Bruno (6-8 capturas)

- Health check funcionando
- Crear juego (POST) con respuesta 201
- Listar juegos (GET) con array de resultados
- Listar con filtros (GET con query params)
- Editar juego (PUT) con prioridad recalculada
- Marcar completado (PATCH) con fecha automática
- Eliminar juego (DELETE)
- Error de validación (POST con body vacío → 400 con errores)

12.5 Servidor corriendo

- Terminal con `npm run dev` mostrando los logs de Morgan

Fin del Sprint 1 — API REST funcional para gestionar backlog de videojuegos.

Siguiente: Sprint 2 — Autenticación con JWT y soporte multi-usuario.